



Redes Neurais Artificiais: *Adaline*

Prof.: Arismar Moraes Gonçalves Júnior

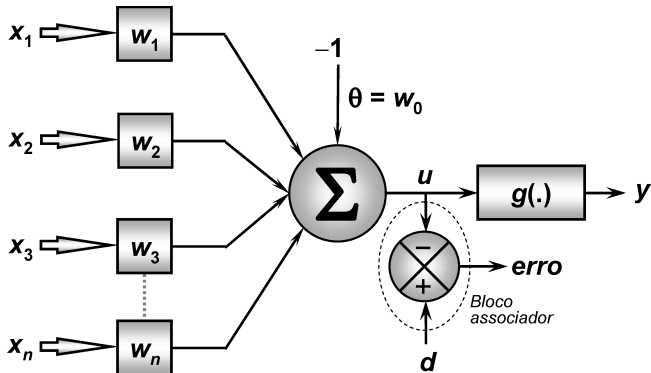
**Engenharia da Computação
Universidade do Estado de Minas Gerais
Unidade Divinópolis**

Divinópolis, Abril de 2026

1. Introdução
2. Princípio de Funcionamento
3. Treinamento
4. Pseudocódigos
5. Comparação *Adaline* x *Perceptron*
6. Aspectos Importantes

- ▶ *Adaline (Adaptive Linear Element):*
 - ▶ Idealizada por Widrow e Hoff em 1960;
 - ▶ Rede simples, mas que promoveu alguns avanços essenciais para a área das RNAs;
 - ▶ Desenvolvimento do algoritmo de aprendizado regra Delta.
- ▶ Características:
 - ▶ Arquitetura *feedforward*;
 - ▶ Utilizada em problemas de classificação envolvendo apenas duas classes;
 - ▶ Empregada em problemas linearmente separáveis;
 - ▶ Treinamento supervisionado.

► Ilustração da rede *Adaline*:



- ▶ Similarmente ao *Perceptron*, pode ser descrito por:

$$u = \sum_{i=1}^n w_i \cdot x_i - \theta = \sum_{i=0}^n w_i \cdot x_i \quad (1)$$

$$y = g(u) \quad (2)$$

- ▶ De modo geral:
 - ▶ Entradas (x_i) ponderadas pelos pesos sinápticos (w_i);
 - ▶ Valor resultante da composição (u) é repassado como argumento da função de ativação $g(\cdot)$;
 - ▶ A saída (y) é o resultado retornado pela função de ativação.
- ▶ A diferença (com relação ao *Perceptron*) reside na utilização de um sinal de erro no processo de treinamento:

$$erro = d - u \quad (3)$$

- Aspectos dos parâmetros característicos do *Adaline*:

Parâmetro	Variável representativa	Tipo característico
Entradas	x_i (i -ésima entrada)	Reais ou binárias (advindas externamente)
Pesos sinápticos	w_i (associado a x_i)	Reais (iniciados aleatoriamente)
Limiar	θ	Real (iniciado aleatoriamente)
Saída	y	Binária
Função de ativação	$g(\cdot)$	Degrau ou degrau bipolar
Processo de treinamento	-----	Supervisionado
Regra de aprendizado	-----	Regra Delta

- ▶ Treinamento:
 - ▶ Processo de ajuste dos pesos da rede;
 - ▶ Baseado em um conjunto de p amostras/observações do processo a ser mapeado;
 - ▶ Realizado de forma supervisionada, ou seja, para cada amostra dos sinais de entrada, conhece-se a respectiva saída/resposta desejada (d);
 - ▶ Cada uma das duas classes do problema de reconhecimento de padrões é associado a um dos dois valores que a saída pode assumir (0/1 ou $-1/1$).

- ▶ Regra de aprendizado Delta ou método do gradiente descendente:
 - ▶ Minimização do erro médio quadrático entre u e d ;
 - ▶ Obtenção de um $\mathbf{w}^*$ ótimo tal que o erro médio quadrático $E(\mathbf{w}^*)$ sobre todo o conjunto de amostras seja o menor possível.
- ▶ Erro médio quadrático:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^p (d^{(k)} - u)^2 \quad (4)$$

- ▶ Função a ser minimizada (substituição da Eq. (1) em (4)):

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^p \left[d^{(k)} - \left(\sum_{i=1}^n w_i \cdot x_i^{(k)} - \theta \right) \right]^2$$

- ▶ Em termos matriciais:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^p \left[d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta \right) \right]^2 \quad (5)$$

- ▶ Para busca do ótimo ($\mathbf{w}^*$), aplicação do operador gradiente¹ do erro em relação ao vetor $\mathbf{w}$:

$$\nabla E(\mathbf{w}) = \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \sum_{k=1}^p \left[d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta \right) \right] \cdot \left(-\mathbf{x}^{(k)} \right) \quad (6)$$

¹Vetor que aponta na direção de maior crescimento de uma dada função.

- ▶ Retornando a Eq. (1) em (6):

$$\nabla E(\mathbf{w}) = - \sum_{i=1}^p \left(d^{(k)} - u \right) . \mathbf{x}^{(k)} \quad (7)$$

- ▶ Passos para adaptação do vetor $\mathbf{w}$ efetuados em direção oposta ao gradiente em uma taxa η . Logo:

$$\Delta \mathbf{w} = -\eta . \nabla E(\mathbf{w}) = \eta . \sum_{k=1}^p \left(d^{(k)} - u \right) . \mathbf{x}^{(k)} \quad (8)$$

$$\mathbf{w}^{atual} = \mathbf{w}^{anterior} + \eta \sum_{k=1}^p \left(d^{(k)} - u \right) . \mathbf{x}^{(k)} \quad (9)$$

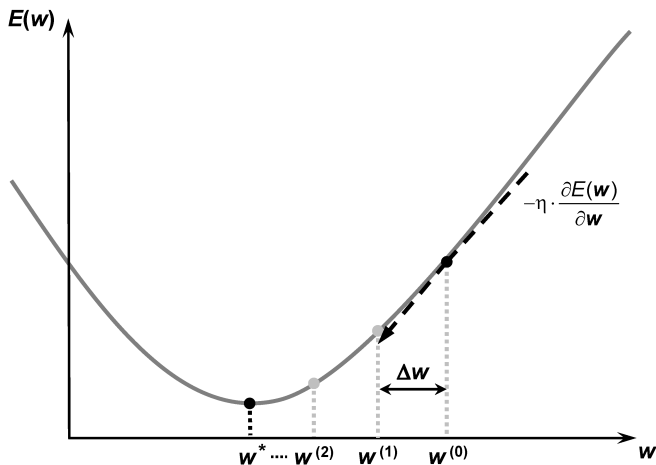
- ▶ Simplificando, a atualização de $\mathbf{w}$ pode ser realizada discretamente após a apresentação de cada k -ésima amostra de treinamento:

$$\mathbf{w}^{atual} = \mathbf{w}^{anterior} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}, \text{ onde } k = 1, \dots, p. \quad (10)$$

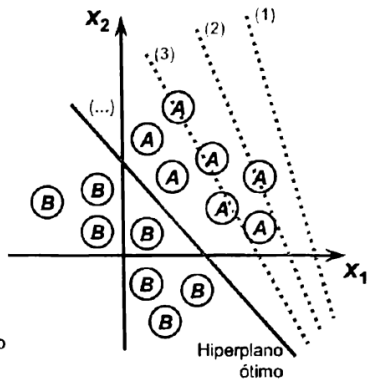
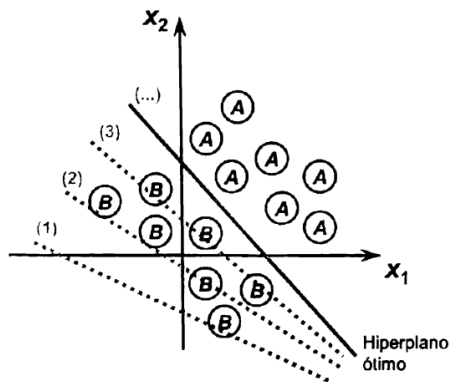
- ▶ Sendo:

- ▶ $\mathbf{w} = [\theta \ w_1 \ w_2 \ \dots \ w_n]^T$ o vetor com o limiar e os pesos;
- ▶ $\mathbf{x}^{(k)} = [-1 \ x_1^{(k)} \ x_2^{(k)} \ \dots \ x_n^{(k)}]^T$ a k -ésima amostra das n entradas do problema em questão;
- ▶ $d^{(k)}$ o valor desejado para a k -ésima amostra de treinamento;
- ▶ u a saída do combinador linear;
- ▶ η a constante que define a taxa de aprendizagem da rede ($0 < \eta < 1$).

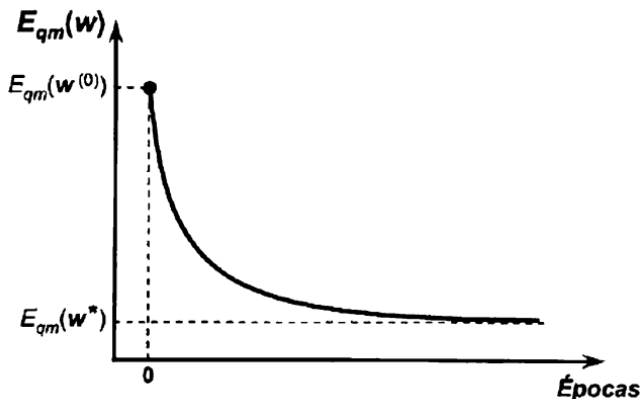
- Ilustração do processo de convergência:



► Ilustração do treinamento:



- Comportamento do erro médio quadrático (E_{qm}) em função do número de épocas de treinamento:



Início {Algoritmo *Adaline* – Fase de Treinamento}

- <1> Obter o conjunto de amostras de treinamento $\{ \mathbf{x}^{(k)} \}$;
- <2> Associar a saída desejada $\{ d^{(k)} \}$ para cada amostra obtida;
- <3> Iniciar o vetor $\mathbf{w}$ com valores aleatórios pequenos;
- <4> Especificar taxa de aprendizagem $\{ \eta \}$ e precisão requerida $\{ \varepsilon \}$;
- <5> Iniciar o contador de número de épocas $\{ \text{época} \leftarrow 0 \}$;
- <6> Repetir as instruções:
 - <6.1> $E_{qm}^{anterior} \leftarrow E_{qm}(\mathbf{w})$;
 - <6.2> Para todas as amostras de treinamento $\{ \mathbf{x}^{(k)}, d^{(k)} \}$, fazer:
 - <6.2.1> $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}^{(k)}$;
 - <6.2.2> $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$;
 - <6.3> $\text{época} \leftarrow \text{época} + 1$;
 - <6.4> $E_{qm}^{atual} \leftarrow E_{qm}(\mathbf{w})$;
- Até que: $|E_{qm}^{atual} - E_{qm}^{anterior}| \leq \varepsilon$

Erro Médio Quadrático:

$$E_{qm}(\mathbf{w}) = \frac{1}{p} \sum_{k=1}^p (d^{(k)} - u)^2$$

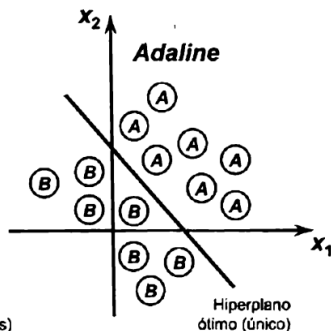
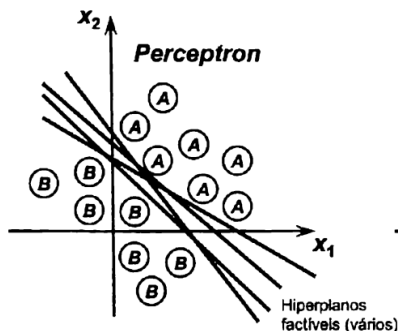
Fim {Algoritmo *Adaline* – Fase de Treinamento}

Início {Algoritmo *Adaline* -- Fase de Operação}

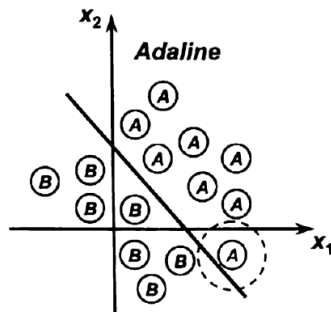
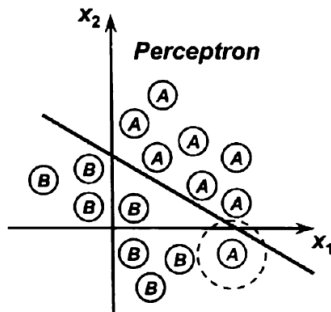
- <1> Obter uma amostra a ser classificada $\{ \mathbf{x} \}$;
- <2> Utilizar o vetor $\mathbf{w}$ ajustado durante o treinamento;
- <3> Executar as seguintes instruções:
 - <3.1> $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}$;
 - <3.2> $y \leftarrow \text{sinal}(u)$;
 - <3.3> Se $y = -1$
 - <3.3.1> Então: amostra $\mathbf{x} \in \{\text{Classe A}\}$
 - <3.4> Se $y = 1$
 - <3.4.1> Então: amostra $\mathbf{x} \in \{\text{Classe B}\}$

Fim {Algoritmo *Adaline* – Fase de Operação}

- ▶ *Adaline* → obtenção de um mesmo hiperplano, independentemente dos valores iniciais dos pesos;
- ▶ *Perceptron* → qualquer hiperplano dentro da faixa de separabilidade é apropriado.



- ▶ O *Adaline* tem maior imunidade a eventuais ruídos que podem afetar o processo a ser mapeado.



- ▶ Diferentemente do *Perceptron*, e dependendo do patamar de precisão (ε) atribuído, a regra Delta pode convergir para problemas não-linearmente separáveis;
- ▶ O parâmetro η deve ser cuidadosamente especificado a fim de evitar instabilidades em torno do ponto de mínimo e também evitar uma convergência lenta;
- ▶ A quantidade de épocas de treinamento varia, de acordo com os valores iniciais dos pesos, disposição espacial das amostras e valor especificado para η ;
- ▶ O desempenho no treinamento é melhorado pela normalização das variáveis de entrada.

- ▶ Silva, I. N.; Spatti, D. H.; Flauzino, R. A. Redes Neurais para engenharia e ciências aplicadas: fundamentos teóricos e aspectos práticos, Editora ArtLiber, 2^a edição, 2016;
- ▶ Haykin, S. Redes Neurais - Principios e Pratica, Bookman, 2^a edição, 2000;.